

C++ Value Categories

Levon Prazyan Nov 2024

Agenda

- Historical overview.
- Value Categories Since c++11
- Value Categories Since c++17
- Special Rules for Value Categories.
- Value Categories and Binding References.
- Lvalue Becomes Rvalue.
- Rvalue Becomes Lvalue.
- Value Categories and *decltype*

Historical Overview

```
int v; // lvalue
v = 4; // OK
4 = v; // ERROR, 4 is an rvalue
```

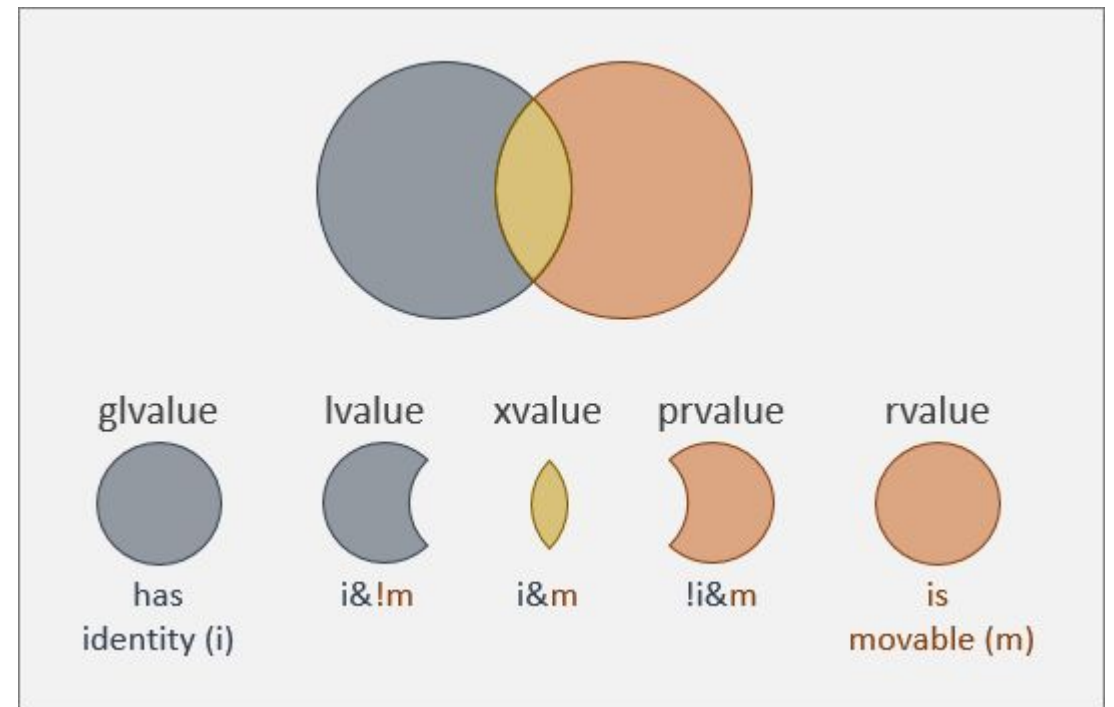
```
const int c = 5; // lvalue or rvalue ?
c = 8;           // ERROR
const int* p = &c; // OK
```

```
std::string s;
std::move(s) = "init"; // OK
auto ps = &std::move(s); // ERROR
```

Value Category	K&R C	ANSI-C	C++98
lvalue	left-hand side of an assignment	locator value	locator value
rvalue	right-hand side of an assignment	readable value	readable value

Value Categories Since c++11

Value Category	C++11
lvalue	locator value
prvalue	pure readable value
xvalue	expiring value
glvalue	generalized lvalue
rvalue	xvalue or prvalue



Value Categories Since c++11 (2)

lvalue examples

- Name of a variable
- String literal
- Return value of a function declared to return a lvalue &
- Any reference to a function
- The result of the built-in unary * operator

prvalue examples

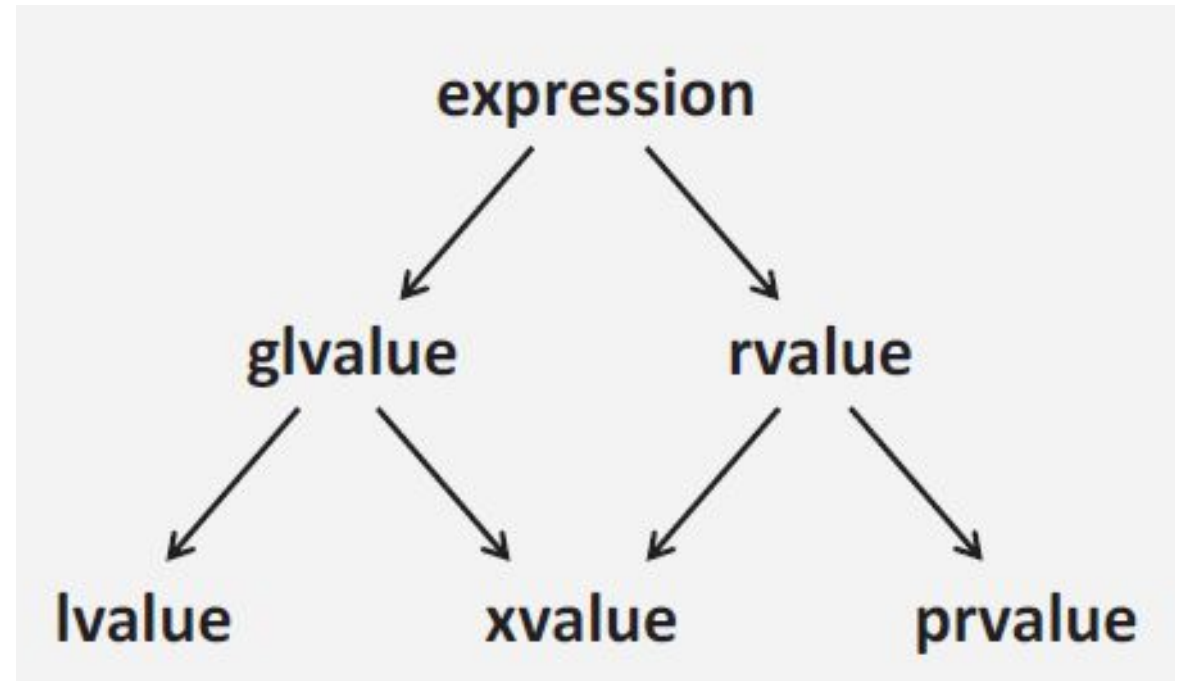
- Built-in literal
- Return type of a function declared to return by value
- Result of the built-in unary & operator
- Lambda expression

xvalue examples

- The result of marking object with std::move
- A cast to a Type&& of an object type
- The return value of a function declared to return Type&&
- A non-static **value** member of a rvalue

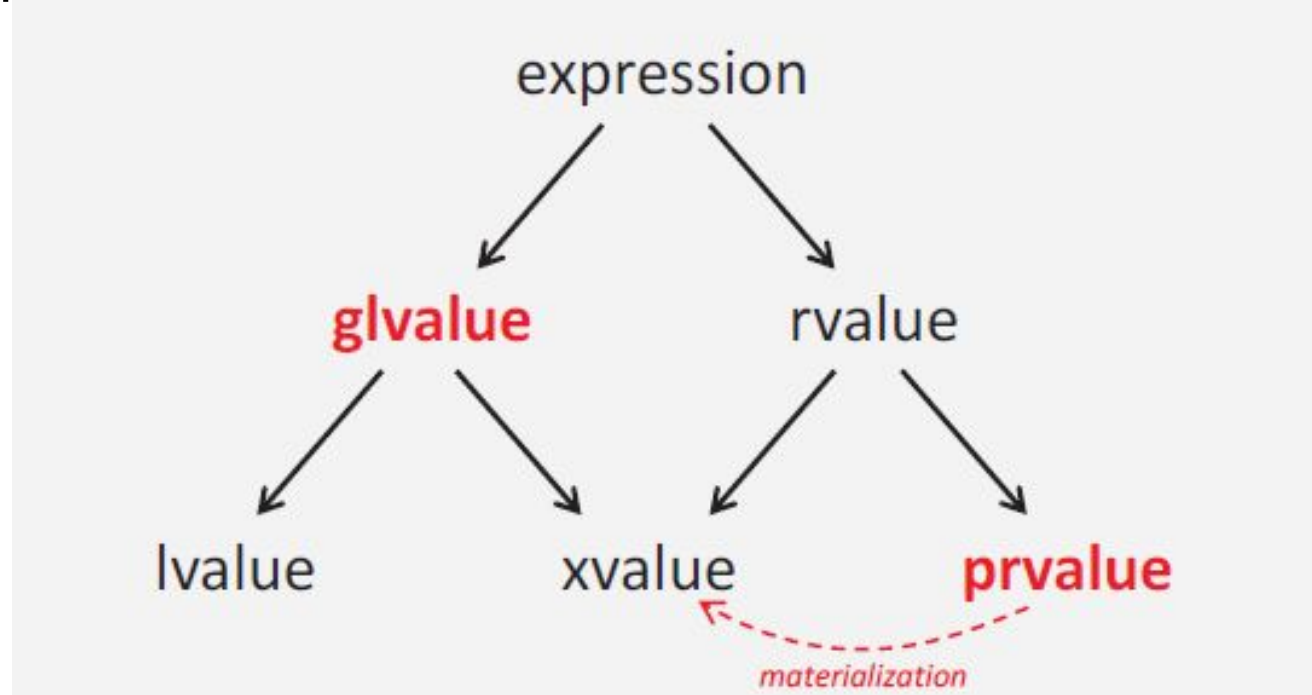
Value Categories Since c++11 (3)

- All names used as expression are **lvalues**.
- All string literals are **lvalues**.
- All non-string literals are **prvalues**.
- All temporaries without a name are **prvalues**.
- All objects marked with `std::move` (including their value members) are **xvalues**.



Value Categories Since c++17

- **glvalues**: expressions for locations of long-living objects or functions.
- **prvalues**: expressions for short living values for initializations.
- **xvalues**: expressions representing a long-living object whose values are no longer needed.
- Temporary materialization conversion is a rvalue to xvalue conversion.



Special Rules for Value Categories

- All expressions that are references to functions are **lvalues**.
 - *void (&&ref)() = f; // ref is a lvalue*
- Data members
 - Data members of lvalues are lvalues.
 - Reference and static data members of rvalues are lvalues
 - Plain data members of rvalues are xvalues.
- *std::move(obj).member* vs *std::move(obj.member)*

Value Categories and Binding References

Call	f(X&)	f(const X&)	f(X&&)	f(const X&&)
f(v)	1	2	-	-
f(c)	-	1	-	-
f(X{})	-	3	1	2
f(std::move(v))	-	3	1	2
f(std::move(c))	-	2	-	1

Universal references are not included in this table.

Value Categories and Binding References

Call	f(X&)	f(const X&)	f(X&&)	f(const X&&)
f(v)	1	2	-	-
f(c)	-	1	-	-
f(X{})	-	3	1	2
f(std::move(v))	-	3	1	2
f(std::move(c))	-	2	-	1

Call	f(X&)	f(const X&)	f(X&&)	f(const X&&)	template <typename T> f(T&&)
f(v)	1	3	-	-	2
f(c)	-	1	-	-	2
f(X{})	-	4	1	3	2
f(std::move(v))	-	4	1	3	2
f(std::move(c))	-	3	-	1	2

Value Categories and Binding References

```
class X {};
```

```
template<typename T>  
void f(T&& arg)  
{ /*...*/ }
```

```
X v;  
const X c;  
f(v);  
f(c);  
f(X{});  
f(std::move(v));  
f(std::move(c));
```

Call	T	arg
f(v)	X&	X&
f(c)	const X&	const X&
f(X{})	X	X&&
f(std::move(v))	X	X&&
f(std::move(c))	const X	const X&&

Value Categories and Binding References,
Lvalue Becomes Rvalue,
Rvalue Becomes Lvalue,
Value Categories and *decltype*
covered by examples

Thank You